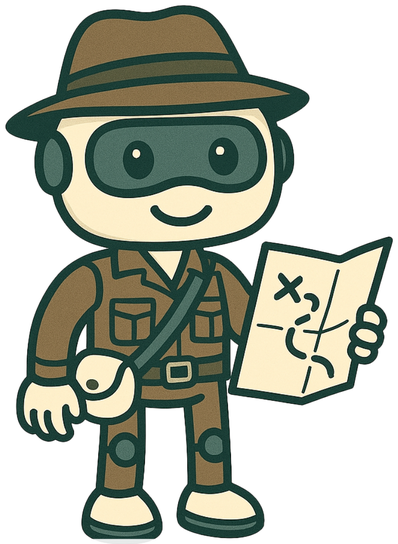
L7: Bandits, Exploration, and Exploitation

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca



So far…

Markov Decision Process

$$\mathscr{M} = (S, A, T, R, \gamma, \mu)$$

Bellman's optimality equations

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

Policy iteration

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_\pi V_k$$

$$\pi'(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^\pi(s') \right]$$

Value iteration

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_* V_k$$

$$\pi^*(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

LP solutions

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a} \quad \text{s.t.} \quad \sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S \qquad x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

HJB Equations

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

POMDPs

$$b_{t+1}(s') = \eta O(o_{t+1} \mid s', a_t) \sum_{s \in S} T(s' \mid s, a_t) b_t(s), \quad \forall s' \in S$$

What critical assumptions have we been making?

- That we know a model of the system, i.e. we have access to exact functions $T(s'|s,a)$, $R(s,a)$.

  ↳ If we remove those assumptions?

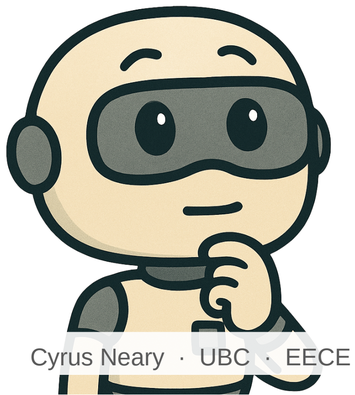
  We now need to approximate $\pi^*(\cdot|s)$ and $V^*(s)$ from data.

model-base RL

First approximate $\hat{T}(s'|s,a)$ and $\hat{R}(s,a)$ and then, use these to estimate $V^*(s)$, and $\pi^*(\cdot|s)$.

model-free RL

Learn estimates of $V^*(s)$ and $\pi^*(\cdot|s)$ directly.



What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

?

$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

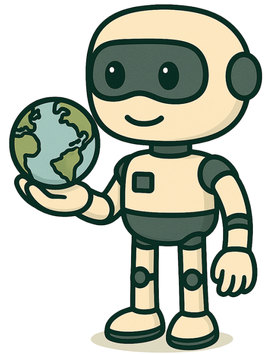
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

More specifically: Design an agent that gathers data from the environment and uses it to estimate π^*, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

?

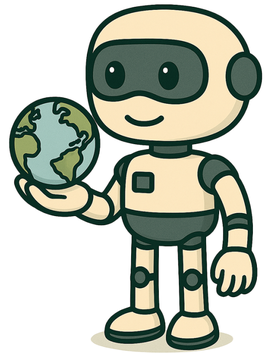
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

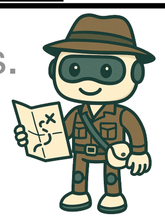
More specifically: Design an agent that gathers data from the environment and uses it to estimate π^*, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

Today's lecture.



What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

?

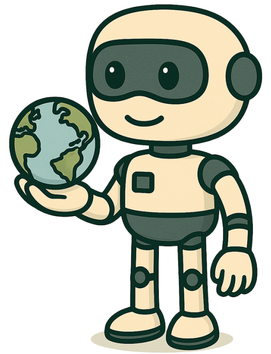
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

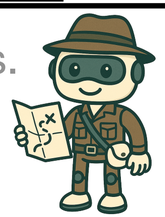
More specifically: Design an agent that gathers data from the environment and uses it to estimate π^*, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

Today's lecture.



How do we use our experience to update approximations of the value function and optimal policy?

Fundamental RL algorithms.

Next few lectures.

What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

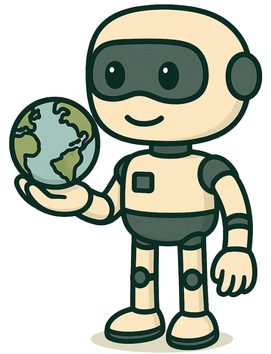
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

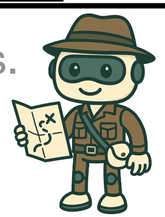
More specifically: Design an agent that gathers data from the environment and uses it to estimate π^*, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

Today's lecture.



How do we use our experience to update approximations of the value function and optimal policy?

Fundamental RL algorithms.

Next few lectures.

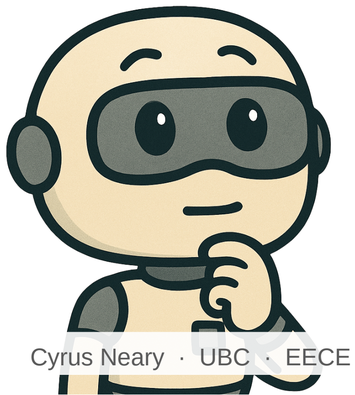
How do we build expressive function classes to help us learn and make decisions from diverse observation (e.g., images, language, etc.)?

Deep RL algorithms.

Part 3 of the course.

When can RL algorithms be useful?

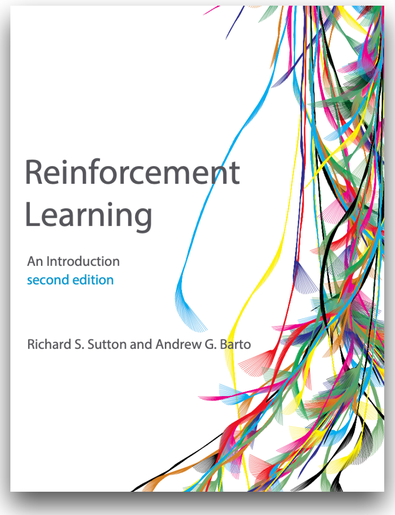
- Complex environments → difficult for direct "traditional" optimization setns.
  - ↳ "pure learning setting": We really don't know the environment /MDP, and need to learn from experience.
  - ↳ "computational setting": We have a good simulator, but exact solutions are hard. We use RL algorithms as a general Approach to approximate dynamic programming
- Even with a perfect sim, RL can be useful to extract/learn a policy.
- Algorithms are general. Can learn good solutions in cases where we have a simulator, but no analytic environment descriptions.



How should an agent explore to gather useful experience?
How should it balance exploration and exploitation?

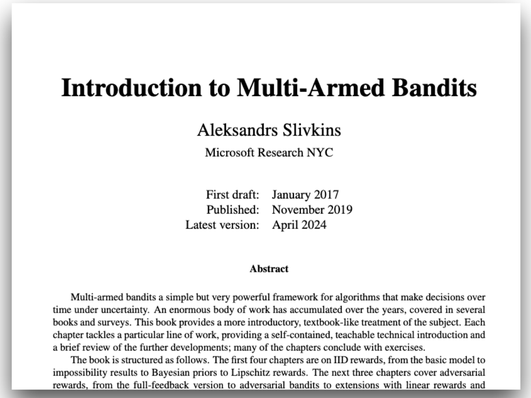
How should an agent explore to gather useful experience?
How should it balance exploration and exploitation?

Today: multi-armed bandits, a problem setting that highlights the analysis of such questions.



Chapter 2: multi-armed bandits

For intuitive descriptions of algorithms and how they relate to ideas in reinforcement learning.



Chapter 1: stochastic bandits

For an introduction to the style of theoretical analysis commonly used in this literature, and further readings.

Today's lecture will draw heavily from chapter 1.

Multi-armed bandits: the problem setting

An agent has K possible actions $A = \{a_1, \ldots, a_K\}$ actions to choose from at each of T decision-making rounds.

Multi-armed bandits: the problem setting

An agent has K possible actions $A = \{a_1, \ldots, a_K\}$ actions to choose from at each of T decision-making rounds.

In each round $t \in \{1,2,\ldots,T\}$, the agent chooses an action $a_t \in A$ and collects an associated reward $r_t \in \mathbb{R}$.

Multi-armed bandits: the problem setting

An agent has K possible actions $A = \{a_1, \ldots, a_K\}$ actions to choose from at each of T decision-making rounds.

In each round $t \in \{1,2,\ldots,T\}$, the agent chooses an action $a_t \in A$ and collects an associated reward $r_t \in \mathbb{R}$.

Each time an action is chosen, the reward is sampled independently from an unknown reward distribution $\mathcal{D}_a$.

Assume per-round rewards are bounded. For simplicity, we could restrict them to the interval $[0,1]$.

Multi-armed bandits: the problem setting

An agent has K possible actions $A = \{a_1, \ldots, a_K\}$ actions to choose from at each of T decision-making rounds.

In each round $t \in \{1,2,\ldots,T\}$, the agent chooses an action $a_t \in A$ and collects an associated reward $r_t \in \mathbb{R}$.

Each time an action is chosen, the reward is sampled independently from an unknown reward distribution $\mathcal{D}_a$.

Assume per-round rewards are bounded. For simplicity, we could restrict them to the interval $[0,1]$.

The agent's goal is to maximize its total reward over the T rounds.

Problem protocol: Stochastic bandits

Parameters: K arms, T rounds (both known); reward distribution $\mathcal{D}_a$ for each arm a (unknown).
In each round $t \in [T]$:

1. Algorithm picks some arm a_t.
2. Reward $r_t \in [0, 1]$ is sampled independently from distribution $\mathcal{D}_a$, $a = a_t$.
3. Algorithm collects reward r_t, and observes nothing else.

From Aleksandrs Slivkins (2024)

Multi-armed bandits: the problem setting

An agent has K possible actions $A = \{a_1, \ldots, a_K\}$ actions to choose from at each of T decision-making rounds.

In each round $t \in \{1,2,\ldots,T\}$, the agent chooses an action $a_t \in A$ and collects an associated reward $r_t \in \mathbb{R}$.

Each time an action is chosen, the reward is sampled independently from an unknown reward distribution $\mathcal{D}_a$.

Assume per-round rewards are bounded. For simplicity, we could restrict them to the interval $[0,1]$.

The agent's goal is to maximize its total reward over the T rounds.

Problem protocol: Stochastic bandits

Parameters: K arms, T rounds (both known); reward distribution $\mathcal{D}_a$ for each arm a (unknown).
In each round $t \in [T]$:

1. Algorithm picks some arm a_t.
2. Reward $r_t \in [0, 1]$ is sampled independently from distribution $\mathcal{D}_a$, $a = a_t$.
3. Algorithm collects reward r_t, and observes nothing else.

From Aleksandrs Slivkins (2024)

Why is it called that??

Name comes from the idea of choosing between a row of slot machines with unknown payouts (each a "one-armed bandit").

Multi-armed bandits: example applications

Ad selection: users visit a webpage, and the agent decides one of many possible ads to display. Each advertisement may be considered an action, and the reward is determined by whether the user clicked on the ad, resulting in a payment to the web advertiser.

Medical trials: a doctor prescribes medical treatments to patients. The choice of treatment may be considered as the action, and the effectiveness of the treatment may be considered the reward.

Investment: At every decision point, the agent chooses one stock to invest in. At the end of a fixed time period, they observe the return on their investment. Their goal is to maximize total wealth.

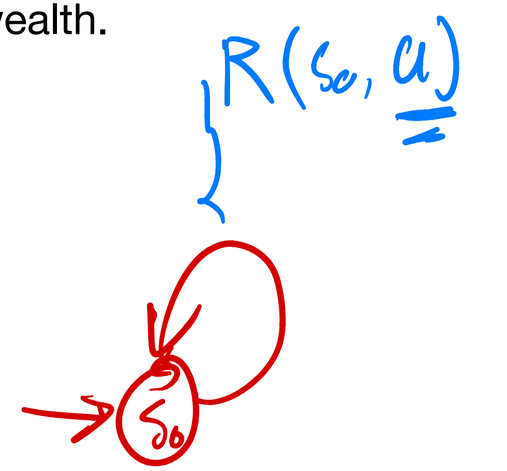
Multi-armed bandits: example applications

Ad selection: users visit a webpage, and the agent decides one of many possible ads to display. Each advertisement may be considered an action, and the reward is determined by whether the user clicked on the ad, resulting in a payment to the web advertiser.

Medical trials: a doctor prescribes medical treatments to patients. The choice of treatment may be considered as the action, and the effectiveness of the treatment may be considered the reward.

Investment: At every decision point, the agent chooses one stock to invest in. At the end of a fixed time period, they observe the return on their investment. Their goal is to maximize total wealth.

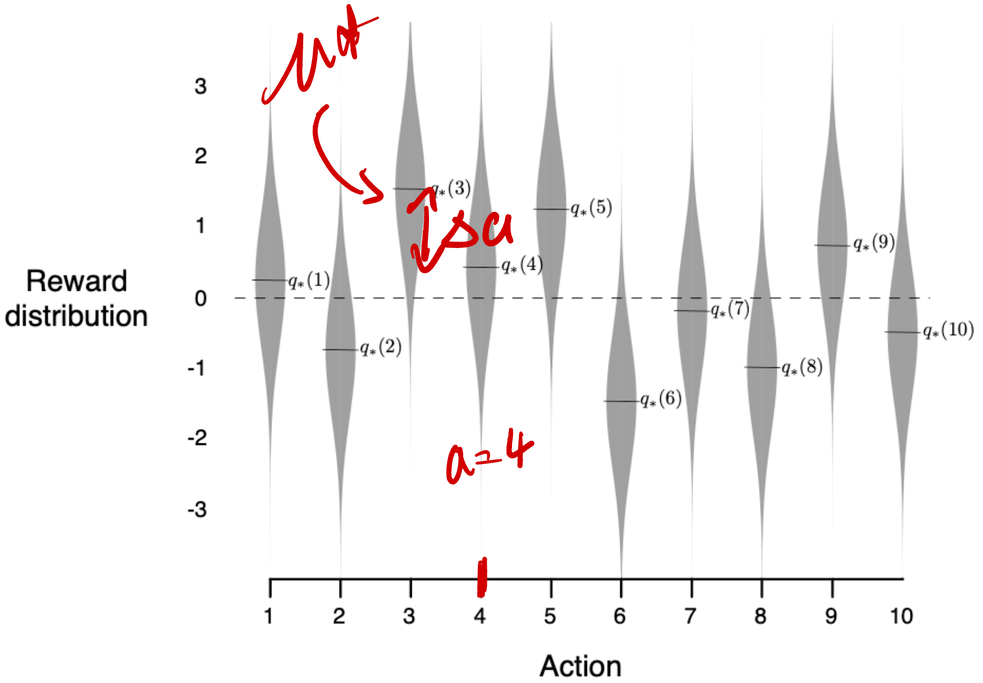
Note: A K-armed bandit problem may be viewed as a one-state MDP with an unknown, stochastic reward function.



Bandit algorithms: definitions

We are interested in the mean reward vector $\mu \in [0,1]^K$, where $\mu(a) = \mathbb{E}[\mathcal{D}_a]$ is the mean reward of action a.

An illustration of reward distributions for a $K = 10$ bandit problem.



From Sutton et al. (2018)

The best mean reward is $\mu^* = \max_{a \in A} \mu(a)$.

The gap $\Delta(a) = \mu^* - \mu(a)$ of action a describes how bad action a is compared to μ^*.

An optimal action a is an action with $\mu(a) = \mu^*$.

Balancing exploration and exploitation

Exploration

- The agent doesn't initially know anything about the reward distributions $\mathscr{D}_a$.
- It needs to try out different actions to gather new information.
- If it incorrectly estimates action a_1 to have the highest mean reward, and thus always selects that action, how would it ever learn that another action has a higher mean reward?

Exploitation

- The agent's objective is to maximize cumulative reward.
- Given the information gathered in past rounds, it can make estimates of the mean rewards associated with each action.
- The agent should exploit that available information to try and obtain the highest possible reward in the near term.

Balancing exploration and exploitation

Exploration

- The agent doesn't initially know anything about the reward distributions $\mathscr{D}_a$.
- It needs to try out different actions to gather new information.
- If it incorrectly estimates action a_1 to have the highest mean reward, and thus always selects that action, how would it ever learn that another action has a higher mean reward?

Exploitation

- The agent's objective is to maximize cumulative reward.
- Given the information gathered in past rounds, it can make estimates of the mean rewards associated with each action.
- The agent should exploit that available information to try and obtain the highest possible reward in the near term.

How do we design agents that intelligently balance exploration and exploitation throughout the decision-making rounds?

How do we evaluate the performance of a decision-making agent in this problem setting?

Evaluating algorithm performance via regret

At round T, how much does the agent “regret” not knowing the best action to take in advance?

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

Evaluating algorithm performance via regret

At round T, how much does the agent “regret” not knowing the best action to take in advance?

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

Best action benchmark. The highest achievable reward.

Evaluating algorithm performance via regret

At round T, how much does the agent “regret” not knowing the best action to take in advance?

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

Best action benchmark. The highest achievable reward.

The cumulative expected reward achieved by the agent, given its action selections a_t.

Evaluating algorithm performance via regret

At round T, how much does the agent "regret" not knowing the best action to take in advance?

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

Best action benchmark. The highest achievable reward.

The cumulative expected reward achieved by the agent, given its action selections a_t.

Note that $R(T)$ is a random variable.

We generally care about the expected regret $\mathbb{E}[R(T)]$, and its dependence on the time horizon T, the number of arms K, and the mean rewards μ.

A uniform exploration algorithm

Idea: Explore arms uniformly, then after a fixed number of exploration rounds, exploit the best action.

1. (Explore) try each action N times.
2. For each action a compute the average reward during exploration $\hat{\mu}(a) = \sum_{i=1}^{N} \frac{X_{a,i}}{N}$.
3. (Exploit) Select the best arm $\hat{a} = \max_{a \in A} \hat{\mu}(a)$, and play it in all remaining rounds.

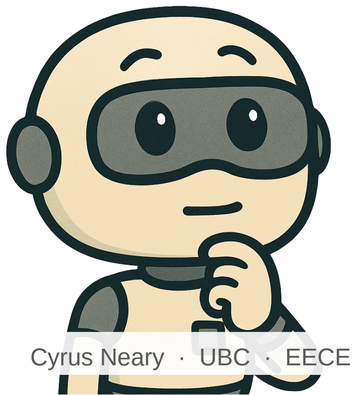
should have $KN < T$

A uniform exploration algorithm

Idea: Explore arms uniformly, then after a fixed number of exploration rounds, exploit the best action.

1. (Explore) try each action N times.
2. For each action a compute the average reward during exploration $\hat{\mu}(a) = \sum_{i=1}^{N} \frac{X_{a,i}}{N}$.
3. (Exploit) Select the best arm $\hat{a} = \max_{a \in A} \hat{\mu}(a)$, and play it in all remaining rounds.

What is the expected regret of this algorithm? How can we start reasoning about algorithm performance?



Recall: Hoeffding's inequality

Statement [edit]

Let $X_1, ..., X_n$ be independent random variables such that $a_i \leq X_i \leq b_i$ almost surely. Consider the sum of these random variables,

$$S_n = X_1 + \cdots + X_n.$$

Then Hoeffding's theorem states that, for all $t > 0$,[3]

$$\mathrm{P}(S_n - \mathrm{E}[S_n] \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$$

$$\mathrm{P}(|S_n - \mathrm{E}[S_n]| \geq t) \leq 2\exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$$

The double sided bound we will use.

Here $\mathrm{E}[S_n]$ is the expected value of S_n.

Note that the inequalities also hold when the X_i have been obtained using sampling without replacement; in this case the random variables are not independent anymore. A proof of this statement can be found in Hoeffding's paper. For slightly better bounds in the case of sampling without replacement, see for instance the paper by Serfling (1974).

From wikipedia

Recall: Hoeffding's inequality

Statement [edit]

Let $X_1, ..., X_n$ be independent random variables such that $a_i \leq X_i \leq b_i$ almost surely. Consider the sum of these random variables,

$$S_n = X_1 + \cdots + X_n.$$

Then Hoeffding's theorem states that, for all $t > 0$,[3]

$$\mathrm{P}(S_n - \mathrm{E}[S_n] \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$$

$$\mathrm{P}(|S_n - \mathrm{E}[S_n]| \geq t) \leq 2\exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$$

The double sided bound we will use.

Here $\mathrm{E}[S_n]$ is the expected value of S_n.

Note that the inequalities also hold when the X_i have been obtained using sampling without replacement; in this case the random variables are not independent anymore. A proof of this statement can be found in Hoeffding's paper. For slightly better bounds in the case of sampling without replacement, see for instance the paper by Serfling (1974).

From wikipedia

For simplicity, consider setting $a_i = 0$ and $b_i = 1$. We are also interested in average returns $\hat{\mu} = \frac{1}{n}S_n = \frac{X_1 + \ldots + X_n}{n}$.

Note that $P(|\hat{\mu} - \mu| \geq \epsilon) = P(|S_n - \mathbb{E}[S_n]| \geq n\epsilon)$.

Setting $t = n\epsilon$ in the above inequality, and noting that $\sum_{i=1}^{n}(b_i - a_i) = n$, we get the following form of the bound.

$$P(|\hat{\mu} - \mu| \geq \epsilon) \leq 2e^{-2n\epsilon^2}$$

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Let's use Hoeffding's inequality: $P(|\hat{\mu}_t(a) - \mu(a)| \geq \epsilon) \leq 2e^{-2n_t(a)\epsilon^2}$

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Pick δ

Let's use Hoeffding's inequality: $P(|\hat{\mu}_t(a) - \mu(a)| \geq \epsilon) \leq 2e^{-2n_t(a)\epsilon^2} \leq \delta$

Choose an ϵ such that $2e^{-2n_t(a)\epsilon^2} \leq \delta$

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Let's use Hoeffding's inequality: $P(|\hat{\mu}_t(a) - \mu(a)| \geq \epsilon) \leq 2e^{-2n_t(a)\epsilon^2}$

Choose an ϵ such that $2e^{-2n_t(a)\epsilon^2} \leq \delta$

Solving for ϵ yields: $\epsilon \geq \sqrt{\frac{\ln(2/\delta)}{2n_t(a)}}$

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Let's use Hoeffding's inequality: $P(|\hat{\mu}_t(a) - \mu(a)| \geq \epsilon) \leq 2e^{-2n_t(a)\epsilon^2}$

Choose an ϵ such that $2e^{-2n_t(a)\epsilon^2} \leq \delta$

Solving for ϵ yields: $\epsilon \geq \sqrt{\frac{\ln(\delta/2)}{2n_t(a)}}$ $\ln(2/\delta)$

Picking $\delta = \frac{2}{T^4}$ gives us $\epsilon = \sqrt{\frac{2\ln(T)}{n_t(a)}}$

Clean events and confidence bounds

Let $n_t(a)$ be the number of rounds before t in which action a was chosen.

Let $\hat{\mu}_t(a)$ be the average rewards of the rounds before t in which action a was chosen.

$$\hat{\mu}_t(a) = \frac{X_{a,1} + \ldots + X_{a,n_t(a)}}{n_t(a)}$$

where $X_{a,j}$ denotes the reward received for the j-th time action a was taken before round t.

Let's use Hoeffding's inequality: $P(|\hat{\mu}_t(a) - \mu(a)| \geq \epsilon) \leq 2e^{-2n_t(a)\epsilon^2}$

Choose an ϵ such that $2e^{-2n_t(a)\epsilon^2} \leq \delta$

Solving for ϵ yields: $\epsilon \geq \sqrt{\frac{\ln(\delta/2)}{2n_t(a)}}$

Picking $\delta = \frac{2}{T^4}$ gives us $\epsilon = \sqrt{\frac{2\ln(T)}{n_t(a)}}$

Define this as the **confidence radius** $rad_T(a) = \sqrt{\frac{2\ln(T)}{n_t(a)}}$.

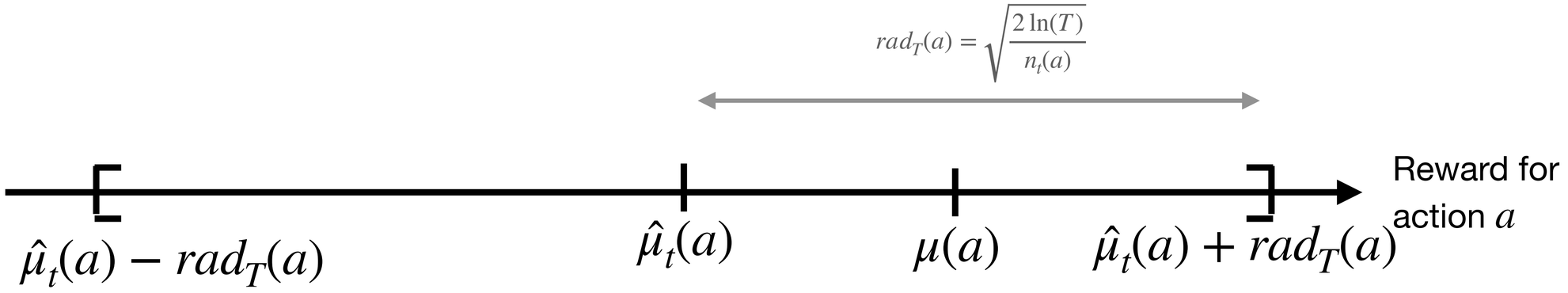
Plugging back into the inequality, we get

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq \sqrt{\frac{2\ln(T)}{n_t(a)}}\right) \geq 1 - \frac{2}{T^4}$$

Interpreting the confidence bound

$$P\left(\left|\hat{\mu}_t(a) - \mu(a)\right| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

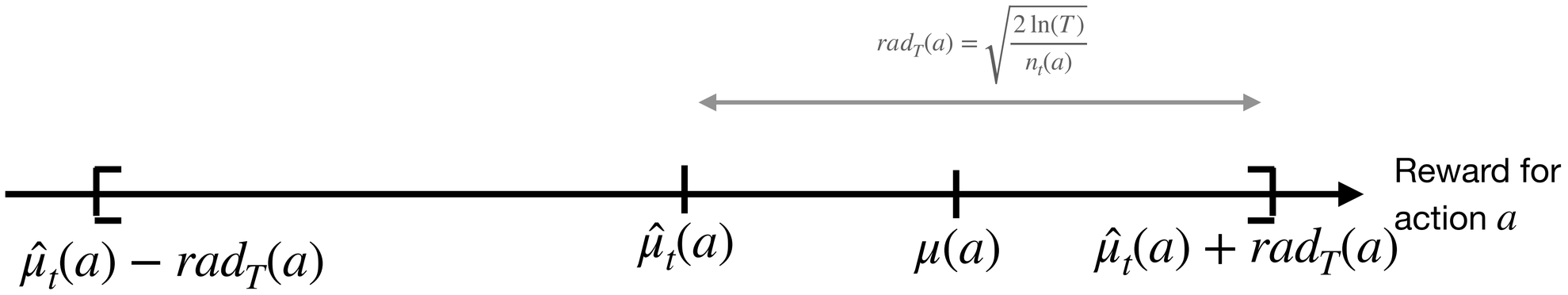
Interpretation: after $n_t(a)$ trials of action a, the true expected reward $\mu(a)$ of action a lies inside the confidence interval $I = [\hat{\mu}(a) - rad_T(a), \hat{\mu}(a) + rad_T(a)]$ with probability of at least $1 - 2/T^4$.



Interpreting the confidence bound

$$P\left(\left|\hat{\mu}_t(a) - \mu(a)\right| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Interpretation: after $n_t(a)$ trials of action a, the true expected reward $\mu(a)$ of action a lies inside the confidence interval $I = [\hat{\mu}(a) - rad_T(a), \hat{\mu}(a) + rad_T(a)]$ with probability of at least $1 - 2/T^4$.



We can thus think of the upper and lower limits of these intervals as defining the:

- *Upper confidence bound*: $UCB_t(a) = \hat{\mu}_t(a) + rad_T(a)$
- *Lower confidence bound:* $LCB_t(a) = \hat{\mu}_t(a) - rad_T(a)$

Aside: visualizing the confidence bound

$$rad_t(a) = \sqrt{\frac{2\ln(t)}{n_t(a)}}$$

- *Upper confidence bound*: $UCB_t(a) = \hat{\mu}_t(a) + rad_t(a)$
- *Lower confidence bound:* $LCB_t(a) = \hat{\mu}_t(a) - rad_t(a)$
- *Confidence Interval:* $CI_t(a) = [LCB_t(a), UCB_t(a)]$

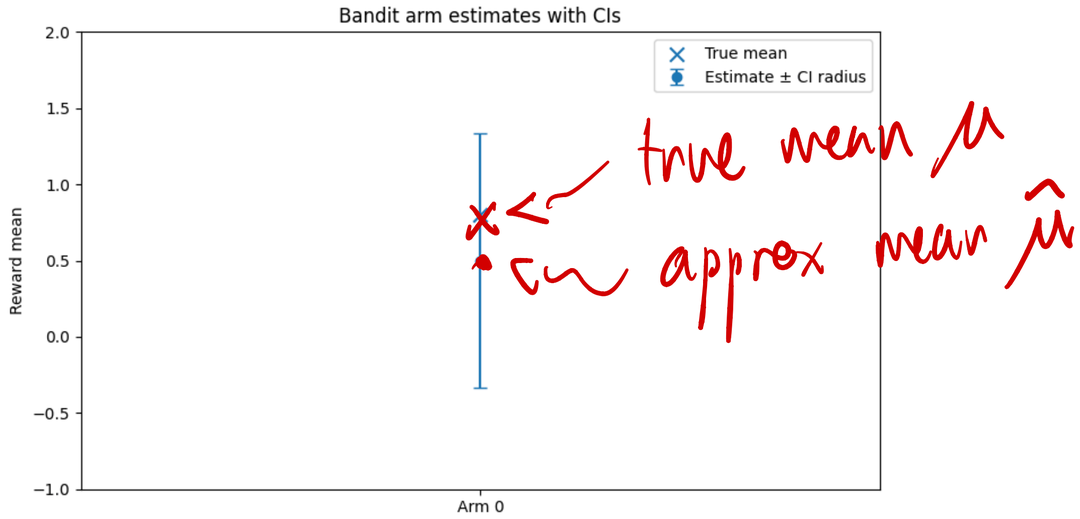
$t = 100$

Aside: visualizing the confidence bound

$$rad_t(a) = \sqrt{\frac{2\ln(t)}{n_t(a)}}$$

- *Upper confidence bound*: $UCB_t(a) = \hat{\mu}_t(a) + rad_t(a)$
- *Lower confidence bound*: $LCB_t(a) = \hat{\mu}_t(a) - rad_t(a)$
- *Confidence Interval*: $CI_t(a) = [LCB_t(a), UCB_t(a)]$

Consider single-arm case with Bernoulli reward distribution with $p = 0.8$.



$t = 2$

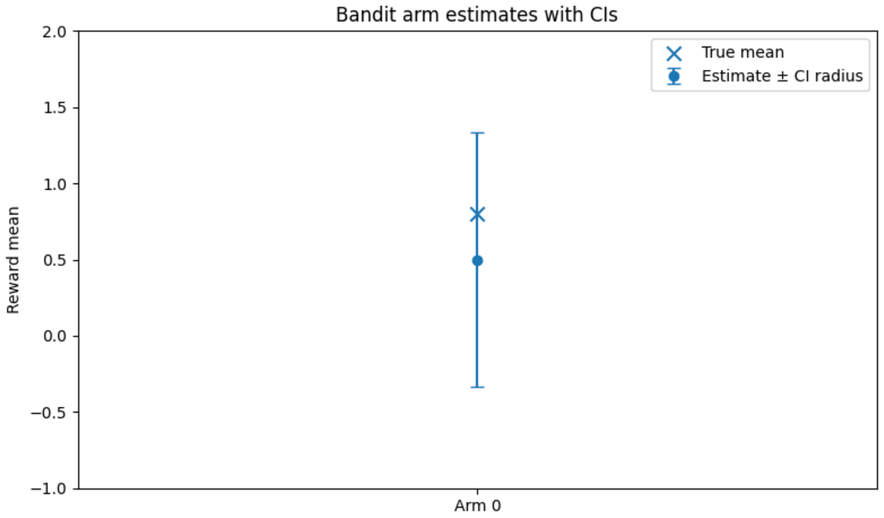
$t = 100$

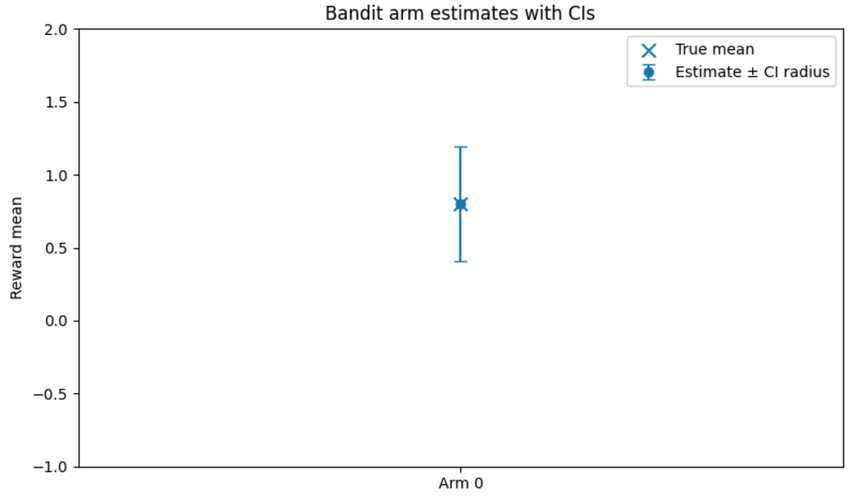
Aside: visualizing the confidence bound

$$rad_t(a) = \sqrt{\frac{2\ln(t)}{n_t(a)}}$$

- *Upper confidence bound*: $UCB_t(a) = \hat{\mu}_t(a) + rad_t(a)$
- *Lower confidence bound*: $LCB_t(a) = \hat{\mu}_t(a) - rad_t(a)$
- *Confidence Interval*: $CI_t(a) = [LCB_t(a), UCB_t(a)]$

Consider single-arm case with Bernoulli reward distribution with $p = 0.8$.





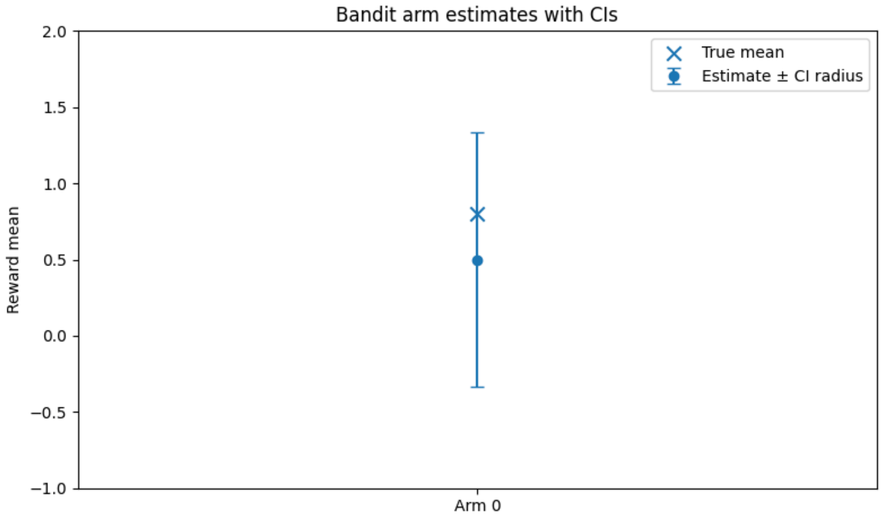
$t = 2$ $t = 50$ $t = 100$

Aside: visualizing the confidence bound

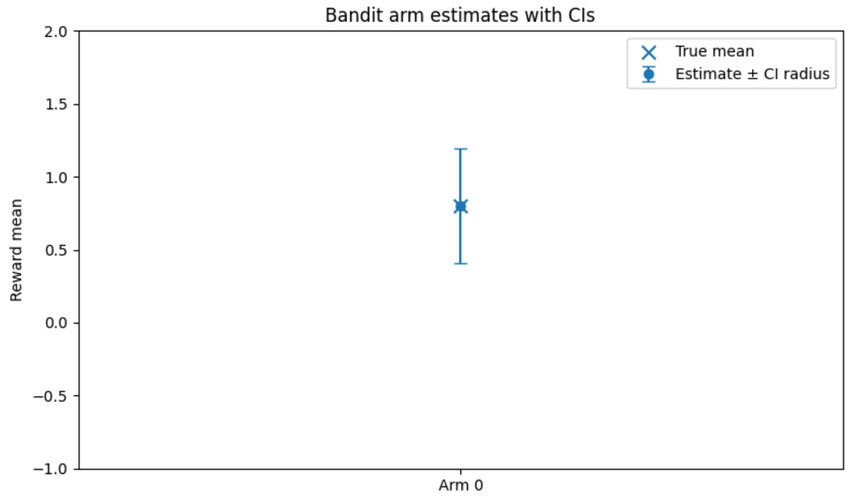
$$rad_t(a) = \sqrt{\frac{2\ln(t)}{n_t(a)}}$$

- *Upper confidence bound*: $UCB_t(a) = \hat{\mu}_t(a) + rad_t(a)$
- *Lower confidence bound*: $LCB_t(a) = \hat{\mu}_t(a) - rad_t(a)$
- *Confidence Interval*: $CI_t(a) = [LCB_t(a), UCB_t(a)]$

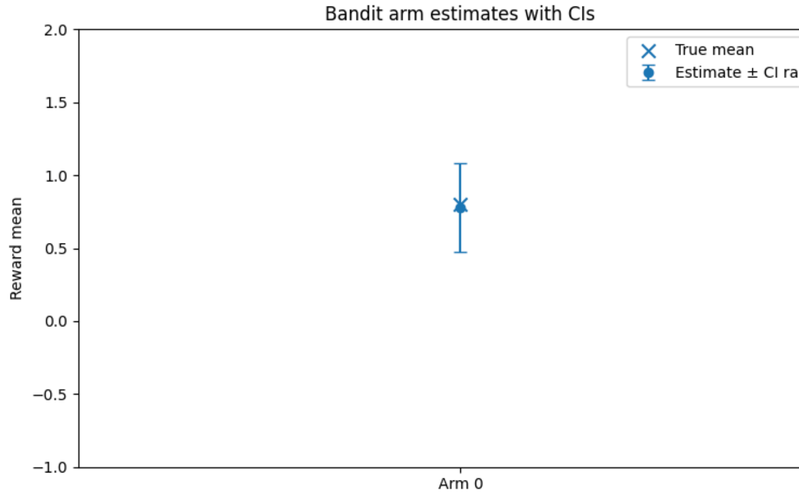
Consider single-arm case with Bernoulli reward distribution with $p = 0.8$.



$t = 2$



$t = 50$

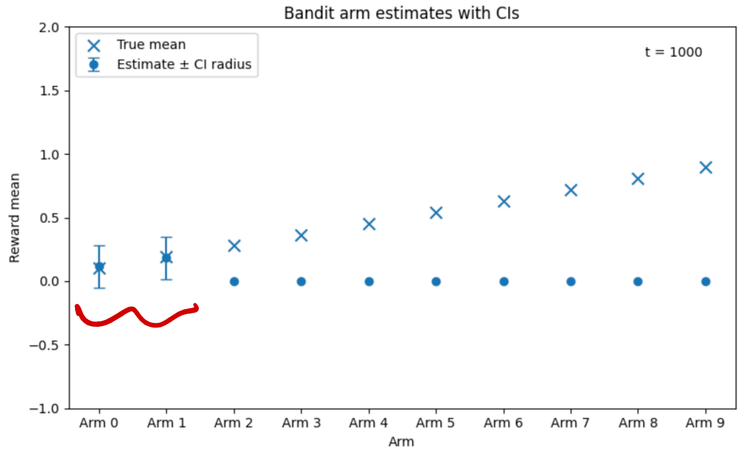


$t = 100$

The uniform exploration algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.

Set exploration budget for each arm to $N = 500$.



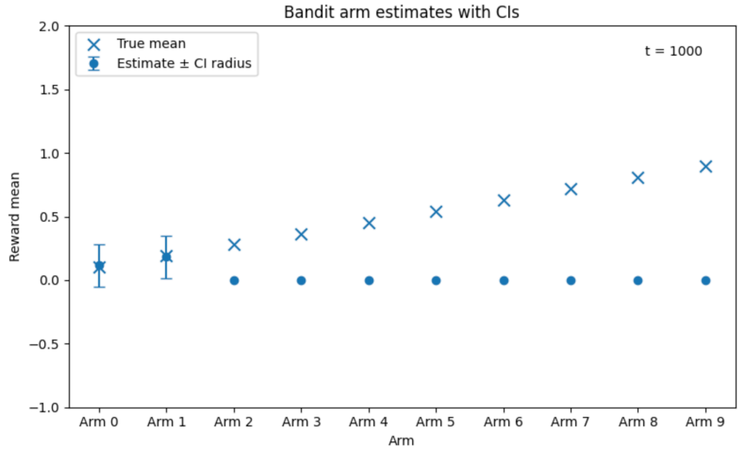
$t = 1000$

Initially, only the first few arms have been attempted since the exploration budget has not yet been exhausted.

The uniform exploration algorithm

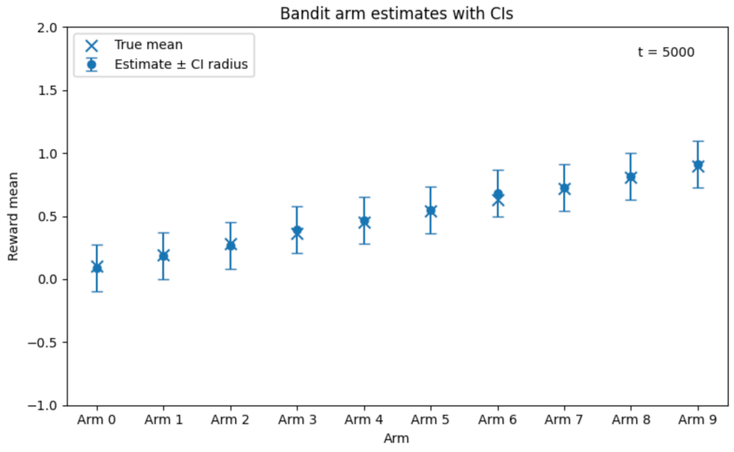
Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.

Set exploration budget for each arm to $N = 500$.



$t = 1000$

Initially, only the first few arms have been attempted since the exploration budget has not yet been exhausted.



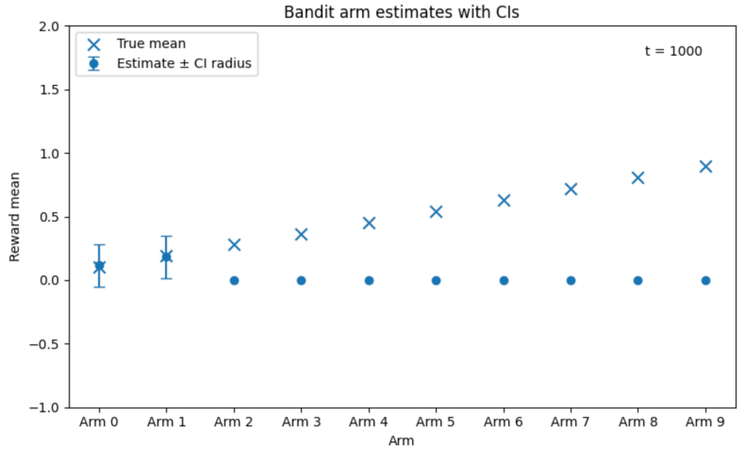
$t = 5000$

After $t = k * N$ steps, exploration budget has been exhausted and the arm with the best estimated reward will be selected thereafter.

The uniform exploration algorithm

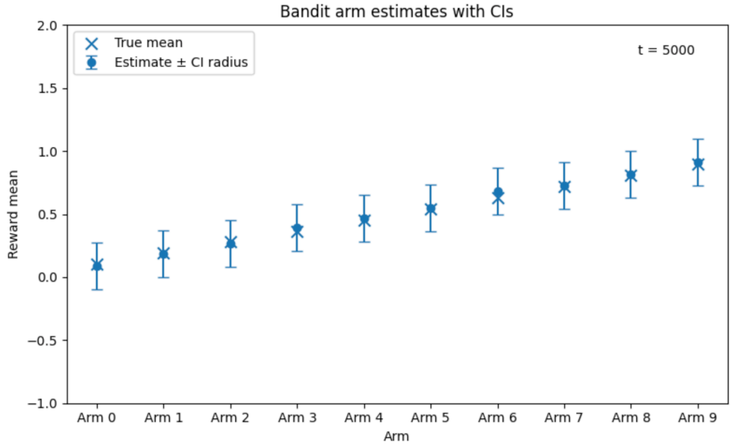
Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.

Set exploration budget for each arm to $N = 500$.



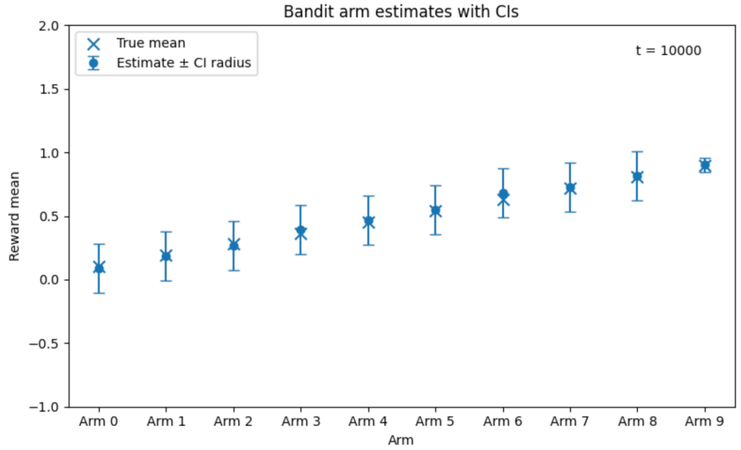
$t = 1000$

Initially, only the first few arms have been attempted since the exploration budget has not yet been exhausted.



$t = 5000$

After $t = k * N$ steps, exploration budget has been exhausted and the arm with the best estimated reward will be selected thereafter.



$t = 10000$

Once this happens, only the estimated best arm is played.

Back to reasoning about regret bounds: defining “clean events”

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Define the *clean event* $\mathscr{E}_{a,t}$ as the event that at round t, for action a, $|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)$.

Back to reasoning about regret bounds: defining “clean events”

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Define the *clean event* $\mathscr{E}_{a,t}$ as the event that at round t, for action a, $|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)$.

Define the *failure event* $\mathscr{F}_{a,t}$, where $|\hat{\mu}_t(a) - \mu(a)| > rad_T(a)$, which has probability $P(\mathscr{F}_{a,t}) \leq \frac{2}{T^4}$.

Back to reasoning about regret bounds: defining “clean events”

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Define the *clean event* $\mathscr{E}_{a,t}$ as the event that at round t, for action a, $|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)$.

Define the *failure event* $\mathscr{F}_{a,t}$, where $|\hat{\mu}_t(a) - \mu(a)| > rad_T(a)$, which has probability $P(\mathscr{F}_{a,t}) \leq \frac{2}{T^4}$.

Now, consider the *global clean event* $\mathscr{E} = \{\forall a \in A,\ \forall t \in T, \quad |\hat{\mu}(a) - \mu(a)| \leq rad_T(a)\} = \bigcap_{a \in A} \bigcap_{t \in [T]} \mathscr{E}_{a,t}$.

Back to reasoning about regret bounds: defining “clean events”

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Define the *clean event* $\mathscr{E}_{a,t}$ as the event that at round t, for action a, $|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)$.

Define the *failure event* $\mathscr{F}_{a,t}$, where $|\hat{\mu}_t(a) - \mu(a)| > rad_T(a)$, which has probability $P(\mathscr{F}_{a,t}) \leq \frac{2}{T^4}$.

Now, consider the g*lobal clean event* $\mathscr{E} = \{\forall a \in A,\ \forall t \in T, \quad |\hat{\mu}(a) - \mu(a)| \leq rad_T(a)\} = \bigcap_{a \in A} \bigcap_{t \in [T]} \mathscr{E}_{a,t}$.

We want to bound the probability of the global clean event. It is related to the probability of individual failure events via De Morgan’s laws:

$$P(\mathscr{E}) = 1 - P(\bar{\mathscr{E}}) = 1 - P\left(\bigcup_{a \in A} \bigcup_{t \in [T]} \mathscr{F}_{a,t}\right)$$

Back to reasoning about regret bounds: defining “clean events”

$$P\left(|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)\right) \geq 1 - \frac{2}{T^4}$$

Define the *clean event* $\mathscr{E}_{a,t}$ as the event that at round t, for action a, $|\hat{\mu}_t(a) - \mu(a)| \leq rad_T(a)$.

Define the *failure event* $\mathscr{F}_{a,t}$, where $|\hat{\mu}_t(a) - \mu(a)| > rad_T(a)$, which has probability $P(\mathscr{F}_{a,t}) \leq \frac{2}{T^4}$.

Now, consider the g*lobal clean event* $\mathscr{E} = \{\forall a \in A,\ \forall t \in T, \quad |\hat{\mu}(a) - \mu(a)| \leq rad_T(a)\} = \bigcap_{a \in A} \bigcap_{t \in [T]} \mathscr{E}_{a,t}$.

We want to bound the probability of the global clean event. It is related to the probability of individual failure events via De Morgan’s laws:

$$P(\mathscr{E}) = 1 - P(\bar{\mathscr{E}}) = 1 - P\left(\bigcup_{a \in A} \bigcup_{t \in [T]} \mathscr{F}_{a,t}\right)$$

And by the union bound, and assuming that $K < T$, we thus have

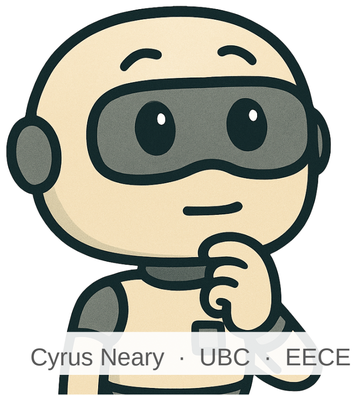
$$P(\mathscr{E}) \geq 1 - \sum_{a \in A} \sum_{t \in [T]} P(\mathscr{F}_{a,t}) \geq 1 - K(\frac{2}{T^3}) \geq 1 - \frac{2}{T^2}$$

A bound on the probability that for every action a taken at every round t of the algorithm’s execution, the true expected reward $\mu(a)$ was within the algorithm’s confidence interval $[LCB_t(a), UCB_t(a)]$.

Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$



Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$

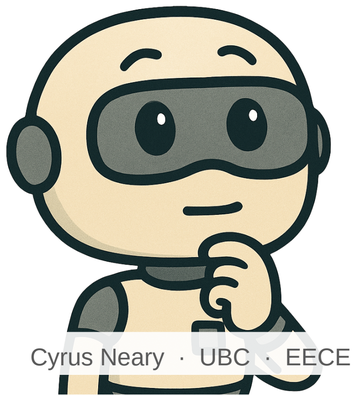
Expected regret

Regret given clean event.

Prob of clean event.

Regret given failure event.

Prob of failure event.



Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$

Expected regret

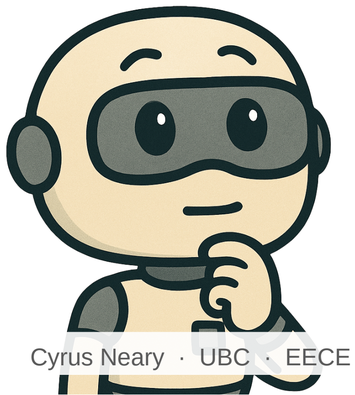
Regret given clean event.

Prob of clean event.

Regret given failure event.

Prob of failure event.

We can bound the regret using the confidence radius $rad_T(a)$.



Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$

Expected regret

Regret given clean event.

Prob of clean event.

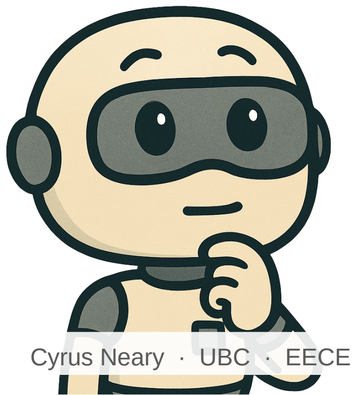
Regret given failure event.

Prob of failure event.

We can bound the regret using the confidence radius $rad_T(a)$.

We have a bound for this from the previous slide.

$$P(\mathscr{E}) \geq 1 - \frac{2}{T^2}$$



Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$

Expected regret

Regret given clean event.

Prob of clean event.

Regret given failure event.

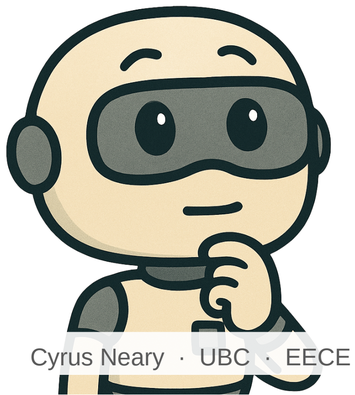
Prob of failure event.

We can bound the regret using the confidence radius $rad_T(a)$.

We have a bound for this from the previous slide.

$$P(\mathscr{E}) \geq 1 - \frac{2}{T^2}$$

This is, in the worst case, bounded by the maximum and minimum rewards for any arm.



Why is it useful to think about clean events and their probabilities?

A useful *rough* strategy for bounding an algorithm's expected regret is as follows:

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T) \,|\, \text{clean event}] \times P(\text{clean event}) + \mathbb{E}[R(T) \,|\, \text{failure event}] \times P(\text{failure event})$$

Expected regret

Regret given clean event.

We can bound the regret using the confidence radius $rad_T(a)$.

Prob of clean event.

We have a bound for this from the previous slide.

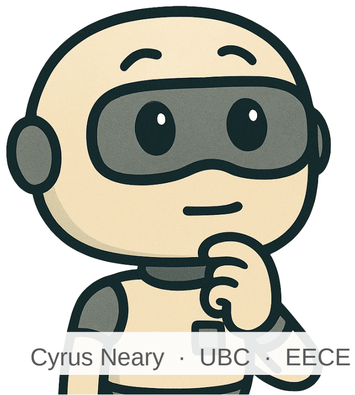
$$P(\mathscr{E}) \geq 1 - \frac{2}{T^2}$$

Regret given failure event.

This is, in the worst case, bounded by the maximum and minimum rewards for any arm.

Prob of failure event.

From our bound, this probability is very small. We can argue that it doesn't contribute much to regret.



Deriving a regret bound for the uniform exploration Alg.

- Exploring each arm with budget N fixed in advance.
- After exploration phase, let $\hat{\mu}(a)$ be the estimated expected reward for a.

$$\hat{\mu}(a) = \frac{1}{N}\sum_{t=1}^{N} r_t(a)$$

- Using the bound from before, $\Pr(|\hat{\mu}(a) - \mu(a)| \leq rad_t(a)) \geq 1 - 2/T^4$

$$rad_t(a) = \sqrt{\frac{2\log(T)}{n_t(a)}} = \sqrt{\frac{2\log(T)}{N}}$$

- So, $\mu(a)$ lives in CI $[\hat{\mu}(a) - rad_t(a), \hat{\mu}(a) + rad_t(a)]$ with high prob.
  - defined the <u>global</u> clean event

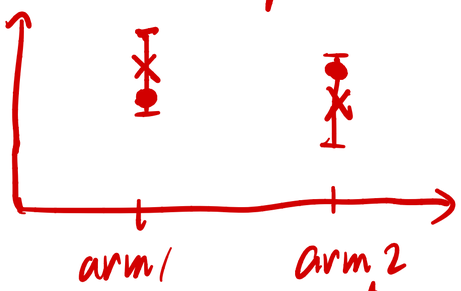
$$\xi = \{\text{event that } \mu(a) \text{ is contained within CI for all arms at every timestep}\}$$

Deriving a regret bound for the uniform exploration Alg.

Case 2 arms Consider only the clean event.

For both arms a_1, a_2, $\mu(a_i) \in [\hat{\mu}(a_i) - \text{rad}(a_i), \hat{\mu}(a_i) + \text{rad}(a_i)]$ for $i \in \{1, 2\}$

Intuition In this case, if we choose the worse arm, it's not so bad because our CIs tell us the expected rewards for those two arms should be close.



$\text{rad}(a) = \text{rad}(a^*) = \text{rad}$

Let a^* be the optimal arm, and let a be the other arm.

If our exploitation phase chooses a over a^*, $\hat{\mu}(a) \geq \hat{\mu}(a^*)$.

Since we're considering $\mathcal{E}$

$$\mu(a) + \text{rad} \geq \hat{\mu}(a) \geq \hat{\mu}(a^*) \geq \mu(a^*) - \text{rad}$$

$$\Rightarrow \mu(a^*) - \mu(a) \leq 2\,\text{rad}$$

Deriving a regret bound for the uniform exploration Alg.

∴ Each round of exploitation contributes <u>at most</u> 2 rad to overall regret.

Each exploration round contributes at most 1 to regret.

So, in the clean event,

$$\text{regret} \le \underbrace{2N}_{\text{exploration}} + \underbrace{2 \times \text{rad} \times (T-2N)}_{\text{exploitation}} \le N + 2 \times \text{rad} \times T$$

Now, we want to pick N that minimizes regret (up to a constant)

Set N, so that $N = 2T \cdot \text{rad}$

$$\Rightarrow N = 2T\sqrt{\frac{\log(T)}{N}} \Rightarrow N^3 = 4T^2 \log(T)$$

$$\Rightarrow \text{set } N = T^{2/3} \log(T)^{1/3} \quad \text{our choice!}$$

$$R(T) \le \text{Constant} \times \left[T^{2/3} \log(T)^{1/3} + 2T\sqrt{\frac{\log T}{T^{2/3} (\log T)^{1/3}}} \right] \Rightarrow R(T) \le O\left(T^{2/3} (\log T)^{1/3}\right)$$

for clean event.

Deriving a regret bound for the uniform exploration Alg.

Consider the "bad" event $\bar{\mathcal{E}} = \{$ there exists some arm a and time t s.t. $\mu(a) \notin [\hat{\mu}(a) - rad, \hat{\mu}(a) + rad]\}$

Regret for a bad event is at worst 1 per round for T rounds.

in Bad event case $R(T) \leq T$

to get to expected regret

$$\mathbb{E}[R(T)] = \mathbb{E}[R(T)|\mathcal{E}]P(\mathcal{E}) + \mathbb{E}[R(T)|\bar{\mathcal{E}}]P(\bar{\mathcal{E}})$$

from our definition of $\mathcal{E}$, and our bounds from last class

$$P(\mathcal{E}) \simeq 1, \quad P(\bar{\mathcal{E}}) \simeq T^{-4}$$

$$\Rightarrow \mathbb{E}[R(T)] \leq O(T^{2/3}(\log T)^{1/3}) \times (\simeq 1) + T \times O(T^{-4})$$

$T \times O(T^{-4}) = 0$ as $T \to \infty$

$$\boxed{\mathbb{E}[R(T)] \leq O(T^{2/3}(\log T)^{1/3})}$$ for two arms

Deriving a regret bound for the uniform exploration Alg.

What about the k-arm case?

Apply the union bound over the k arms to get

$$P(\mathcal{E}) \geq 1 - k\left(\frac{2}{T^4}\right)$$

Regret during exploration is now bound by kN

So, $R(T) \leq Nk + 2 \times rad \times T$ for clean event case.

Setting $Nk = 2 \times rad \times T$

$$E[R(T) \mid \mathcal{E}] \leq O\left(T^{2/3}(k \log(T))^{1/3}\right)$$

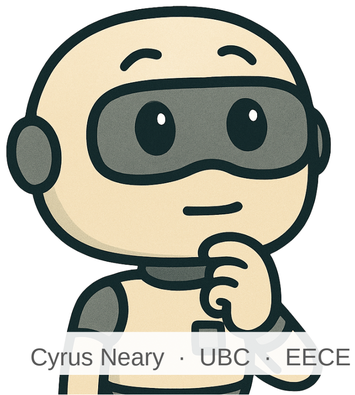
---

use same logic as before to get

$$E[R(T)] \leq O\left(T^{2/3}(k \log(T))^{1/3}\right)$$

Why is a K-armed bandit problem a learning problem rather than a planning problem?

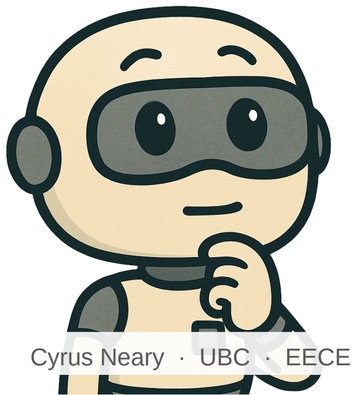
A: Rewards are stochastic, and any stochastic problem requires learning.
B: It has many states with unknown probabilities
C: The horizon T is unknown to the agent a priori.
D: The reward distributions are unknown, so the agent's own choices dictate what information it gets.
E: I don't understand.



Agent 1 picks the optimal arm μ^* every round and collects 70 reward over $T = 100$. Agent 2 sometimes picks suboptimal arms but, by luck, collects 75 reward over the same horizon.

Using $R_2(T) = \mu^* \cdot T - \sum_t \mu_2(a_t)$, which of the following is true?

A: Agent 2 has lower regret because it collected more reward.
B: Both have the same regret.
C: Agent 1 has zero regret.
D: Regret can't be computed without the realized rewards.
E: I don't know what's going on.



Definition of regret depends on expected rewards not realized rewards.

For a fixed T, how many times more must you pull an arm to halve its confidence interval's radius $rad_t(a) = \sqrt{\frac{2ln(t)}{n_t(a)}}$?

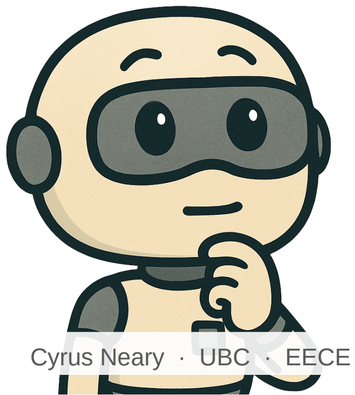
A: $2\times$

B: $4\times$

C: $\sqrt{2}\times$

D: $\ln(2)\times$

E: I don't understand.



The epsilon-greedy algorithm

Idea: with high probability, act in a “greedy” way. Otherwise, explore randomly.

For each round $t = 1,2,\ldots$

With probability ϵ_t:

(Explore) choose an action uniformly at random

With probability $1 - \epsilon_t$:

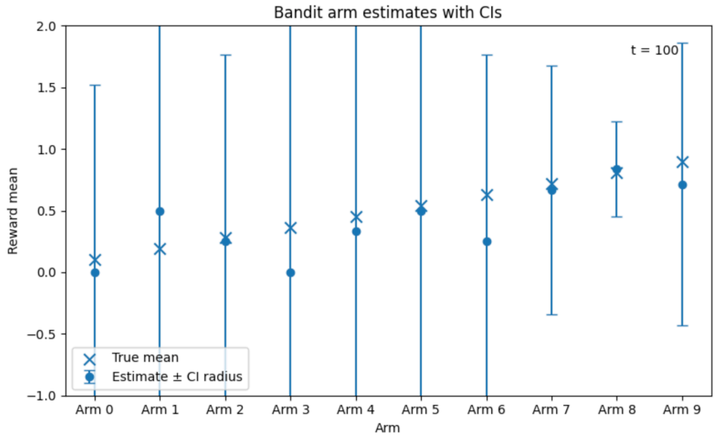
(Exploit) take the action with the highest average reward so far $a_t = \max_{a \in A} \hat{\mu}_t(a)$.

Update the average reward estimate $\hat{\mu}_t(a)$ with the resulting reward.

Note: ϵ-greedy exploration is very frequently used in RL algorithms due to its simplicity.

The epsilon-greedy algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.
Set the exploration parameter to 0.3. $= \epsilon$

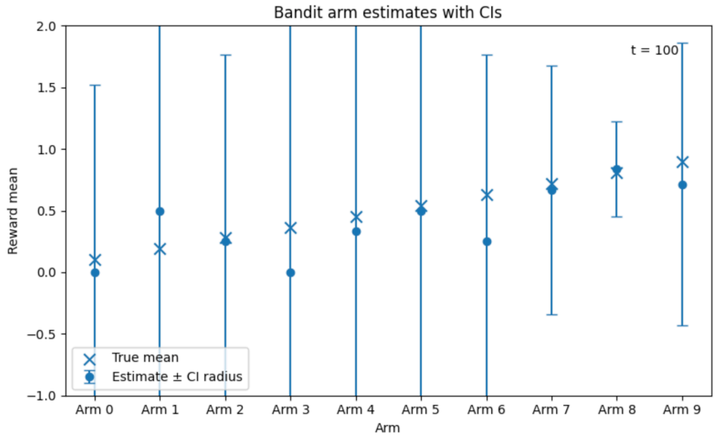


$$t = 100$$

Initially, the best arm is played most of the time. Other arms are selected at random 30% of the time.

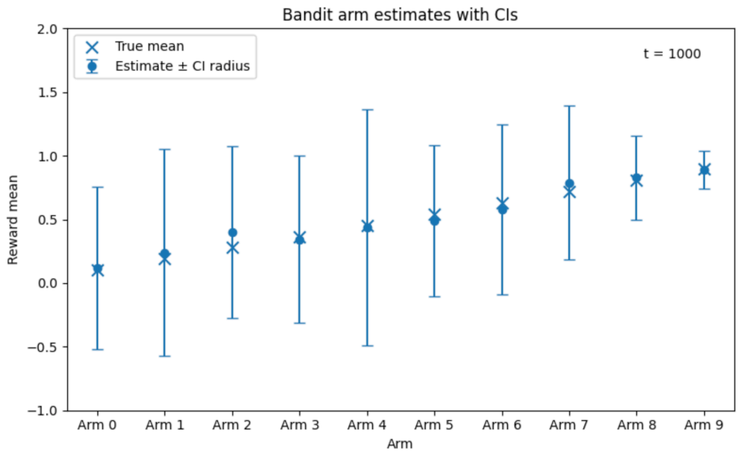
The epsilon-greedy algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9. Set the exploration parameter to 0.3.



$t = 100$

Initially, the best arm is played most of the time. Other arms are selected at random 30% of the time.

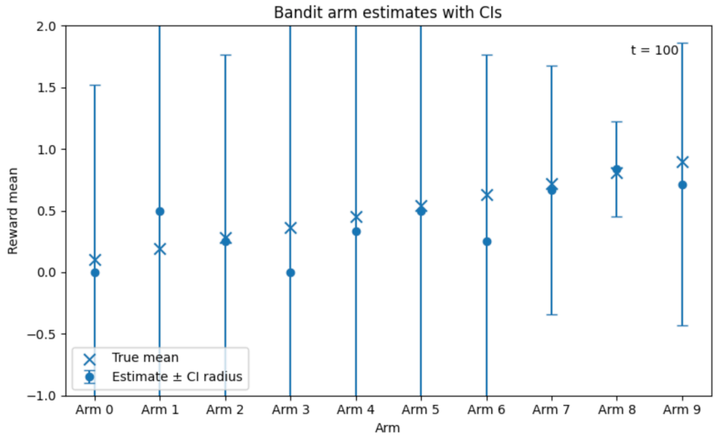


$t = 1000$

After a while, CI of all arms will shrink, and the algorithm's idea of the "best arm" will improve.

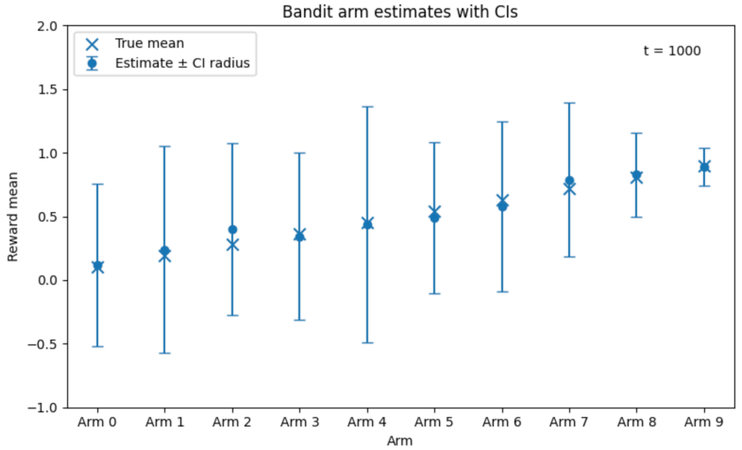
The epsilon-greedy algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.
Set the exploration parameter to 0.3.



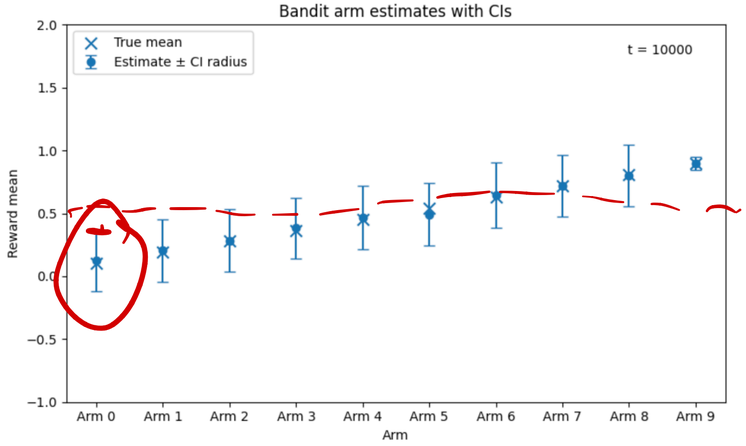
$t = 100$

Initially, the best arm is played most of the time. Other arms are selected at random 30% of the time.



$t = 1000$

After a while, CI of all arms will shrink, and the algorithm's idea of the "best arm" will improve.



$t = 10000$

Even after the best arm is clearly identified, the algorithm will continue to explore uniformly with probability set by parameter ϵ.

Adaptive vs. non-adaptive exploration

Classify a decision round t as an *exploration round* if the data from that round is used by the algorithm in future rounds.

Uniform and ϵ-greedy exploration policies are *non-adaptive*: they fix the number of exploration rounds before the algorithm starts.

Adaptive exploration strategies, which adapt exploration to the observations, so that very underperforming arms are phased out sooner, can achieve better regret bounds.

The successive elimination algorithm

Idea: use confidence bounds to eliminate actions that are clearly worse than others. Keep trying each action a until $UCB_t(a) < LCB_t(a')$ for some other $a' \in A$, at which point a no longer needs to be considered.

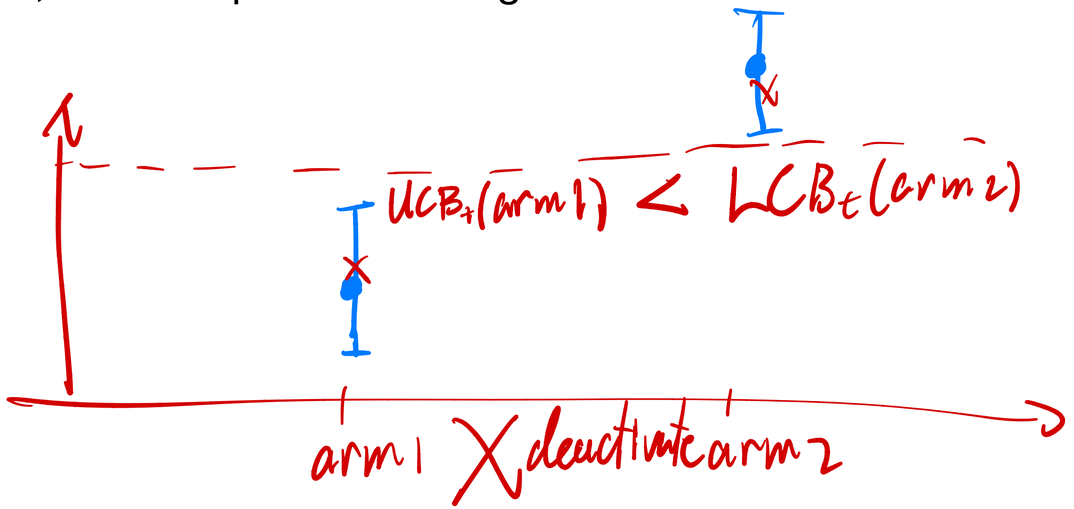
Initialize each action as being *active.*

While true:

Try each active action once.

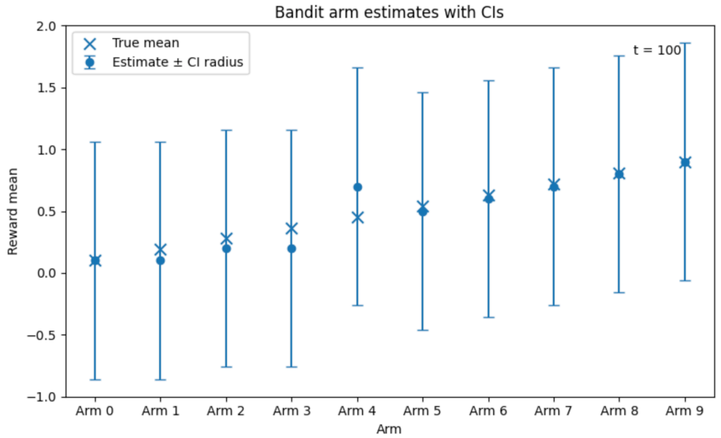
For each action $a \in A$

If $UCB_t(a) < LCB_t(a')$ for some other action $a' \in A$, deactivate action a.



The successive elimination algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.

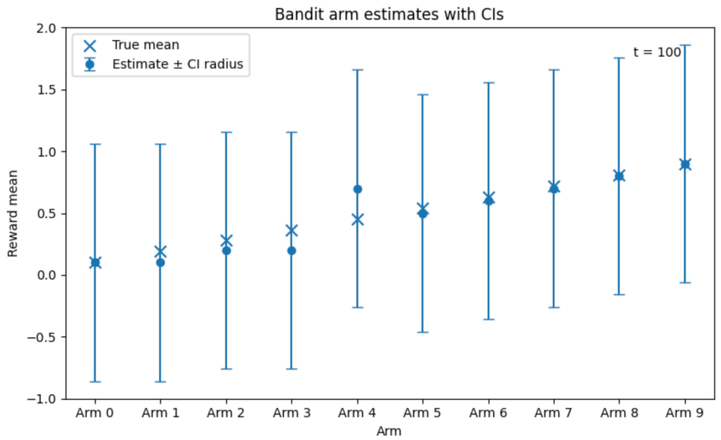


$t = 100$

Initially, CIs are identical for all arms because they are all selected an equal number of times until one is eliminated.

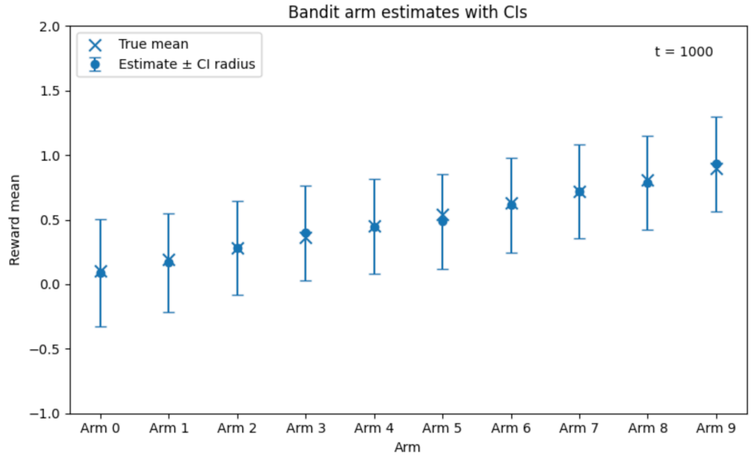
The successive elimination algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.



$t = 100$

Initially, CIs are identical for all arms because they are all selected an equal number of times until one is eliminated.

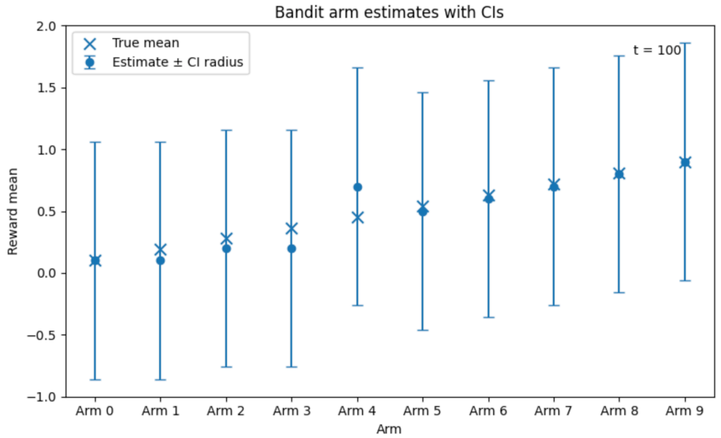


$t = 1000$

This selection strategy continues for some time, until the UCB of one arm is lower than the LCB for some other arm.

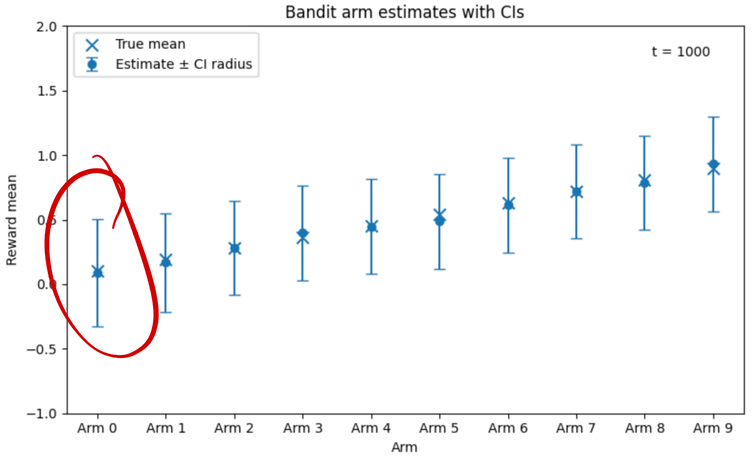
The successive elimination algorithm

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.



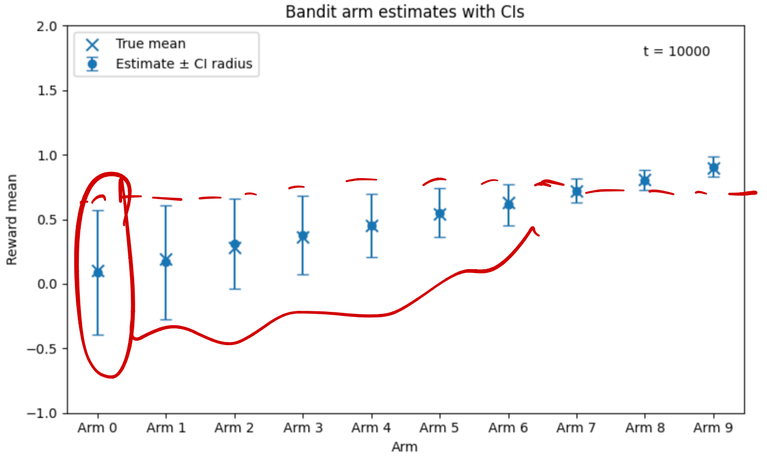
$t = 100$

Initially, CIs are identical for all arms because they are all selected an equal number of times until one is eliminated.



$t = 1000$

This selection strategy continues for some time, until the UCB of one arm is lower than the LCB for some other arm.



$t = 10000$

Once this happens, the arms with low UCBs start to be eliminated, and only the highly performing arms are selected.

UCB1: Optimism under uncertainty

Idea: act *optimistically* as if $UCB_t(a)$ was the true expected reward.

$LCB_t(a)$

Try each action at least once.

For each round $t = 1,2,\ldots T$

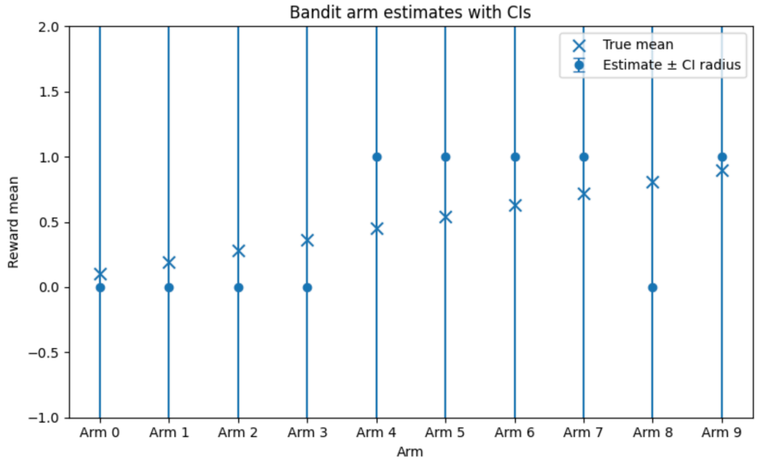
Take action $a \in \arg\max_{a \in A} UCB_t(a)$

Reminder

$$UCB_t(a) = \underbrace{\hat{\mu}_t(a)}_{\text{exploit}} + \underbrace{\sqrt{\frac{2\log T}{n_t(a)}}}_{\text{exploration term}}$$

UCB1: Optimism under uncertainty

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.

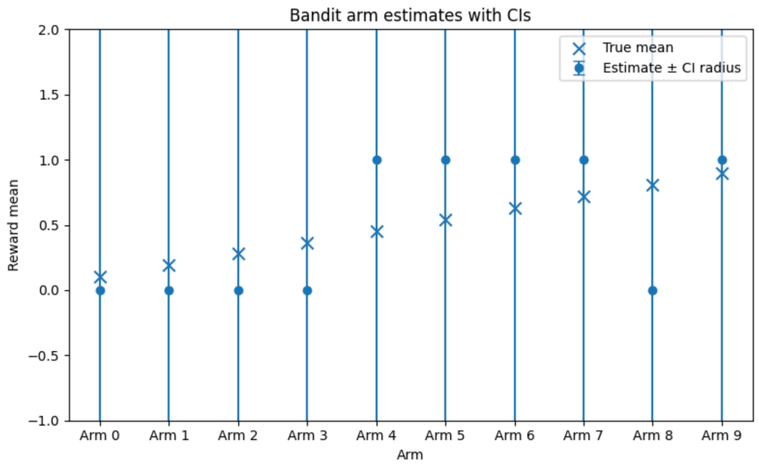


$$t = 10$$

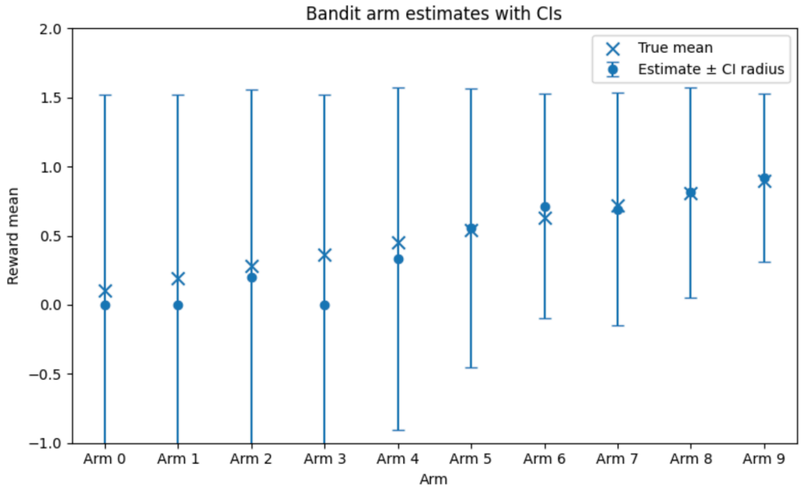
Initially, CIs are similar for all arms because under very limited samples, UCBs are all roughly the same.

UCB1: Optimism under uncertainty

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.



$t = 10$



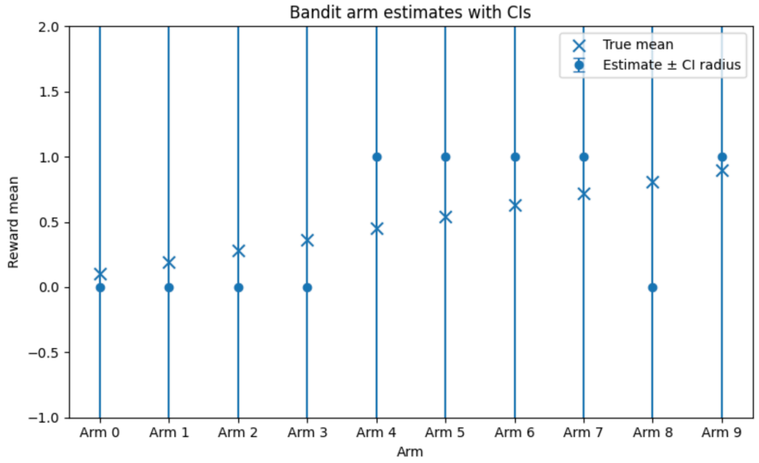
$t = 100$

Initially, CIs are similar for all arms because under very limited samples, UCBs are all roughly the same.

UCB1 chooses the arm with the maximum UCB at each step. So, the UCBs stay relatively consistent (if one was much higher than the others, it would be chosen until the CI shrinks).

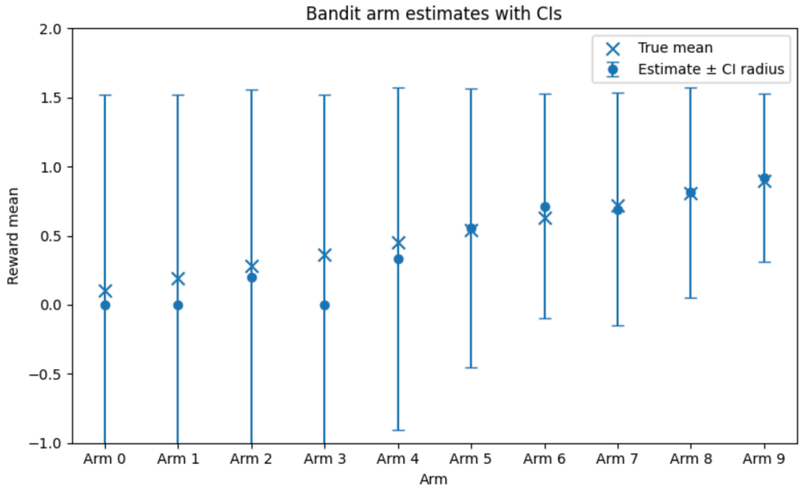
UCB1: Optimism under uncertainty

Consider ten-arm case with Bernoulli reward distributions with p between 0.1 and 0.9.



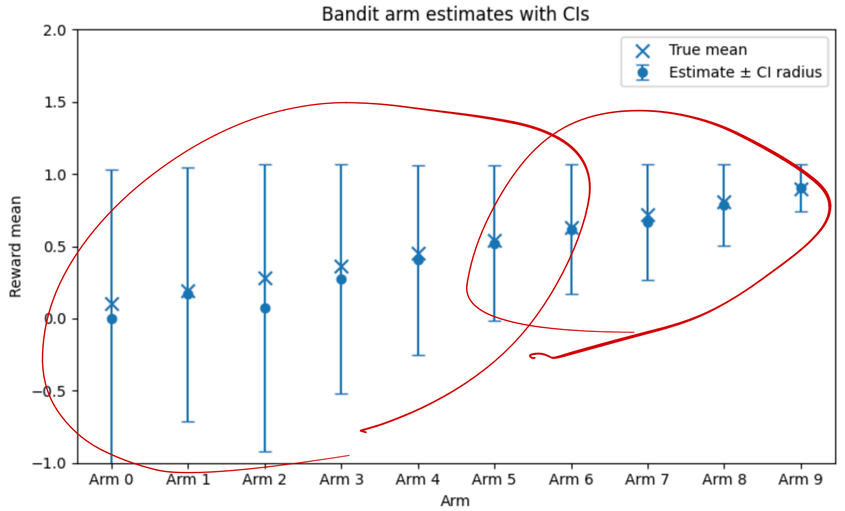
$t = 10$

Initially, CIs are similar for all arms because under very limited samples, UCBs are all roughly the same.



$t = 100$

UCB1 chooses the arm with the maximum UCB at each step. So, the UCBs stay relatively consistent (if one was much higher than the others, it would be chosen until the CI shrinks).



$t = 1000$

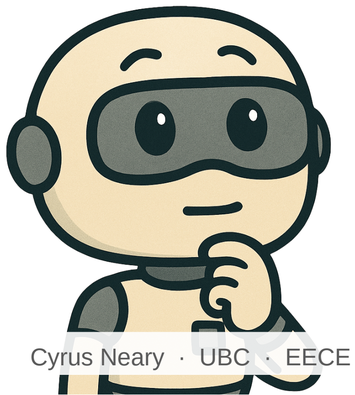
Once the CI shrinks sufficiently, only the arms with high estimated reward values are played.

Why does UCB1 work?

$$UCB_t = \hat{\mu}_t(a) + \sqrt{\frac{2\log(T)}{n_t(a)}}$$

exploit — explore

- Intuitively: Initially, we don't have enough samples, and all our CIs are very large, and we'll be likely to explore.
  After a while we get more samples from playing, confidence intervals shrink, and we naturally tend towards exploitation.



Thompson Sampling

Idea: Keep a posterior belief distribution over each arm's mean. To make decisions, sample each arm's distribution and select the maximum of this set of samples.

In the case the arm rewards are given by Bernoulli random variables with parameters p:

Uniform belief prior for arm i is given by Beta(α_i, β_i)

At each round t:

Sample $\tilde{\mu}_i \sim$ Beta(α_i, β_i) for all arms.

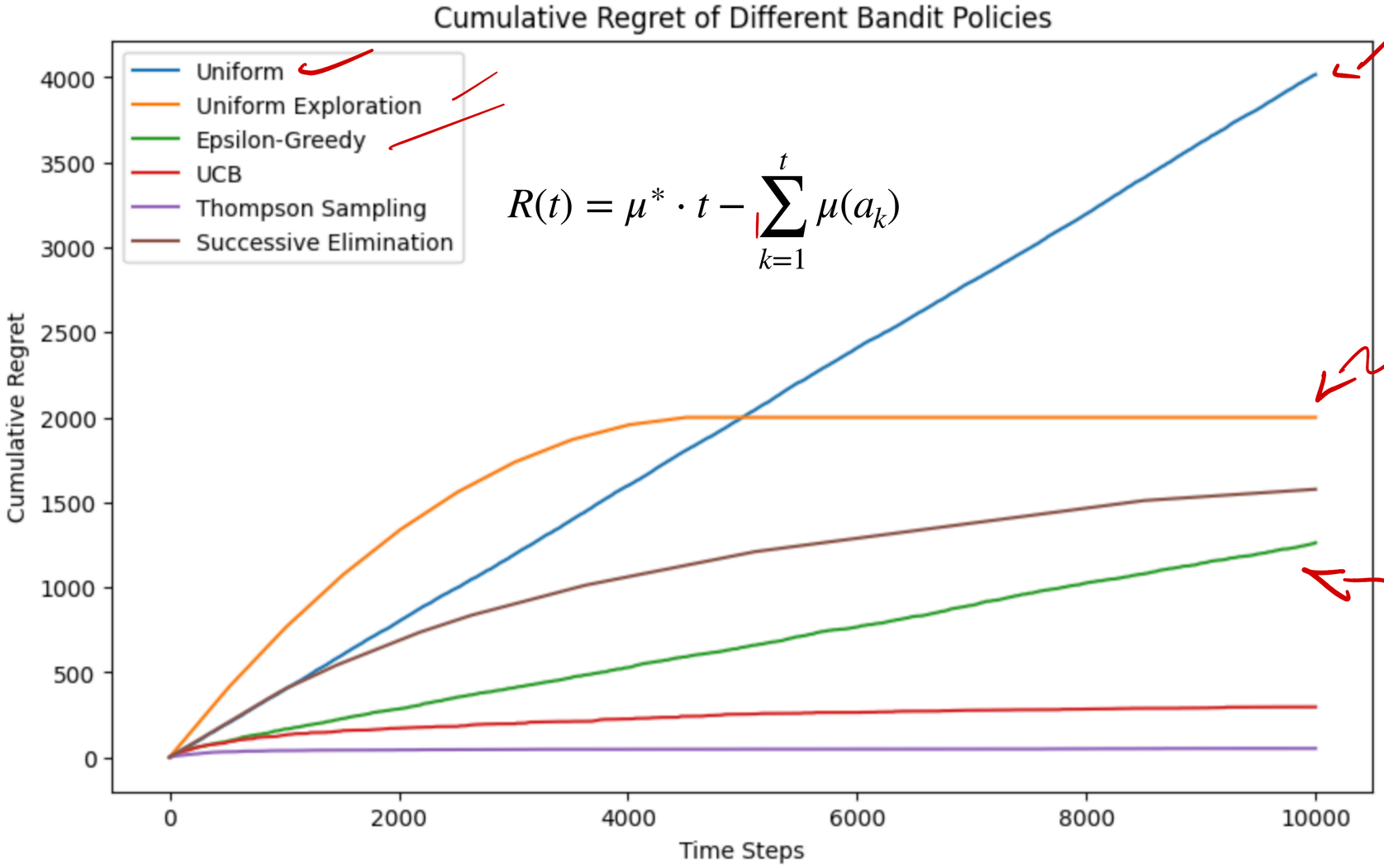
Play $a_t = \arg\max_i \tilde{\mu}_i$

Observe the reward $r_t \in \{0,1\}$

Update the arm's posterior:

$$\alpha_{a_t} \leftarrow \alpha_{a_t} + r_t \qquad \beta_{a_t} \leftarrow \beta_{a_t} + 1 - r_t$$

Comparing cumulative regret for algorithms we've seen

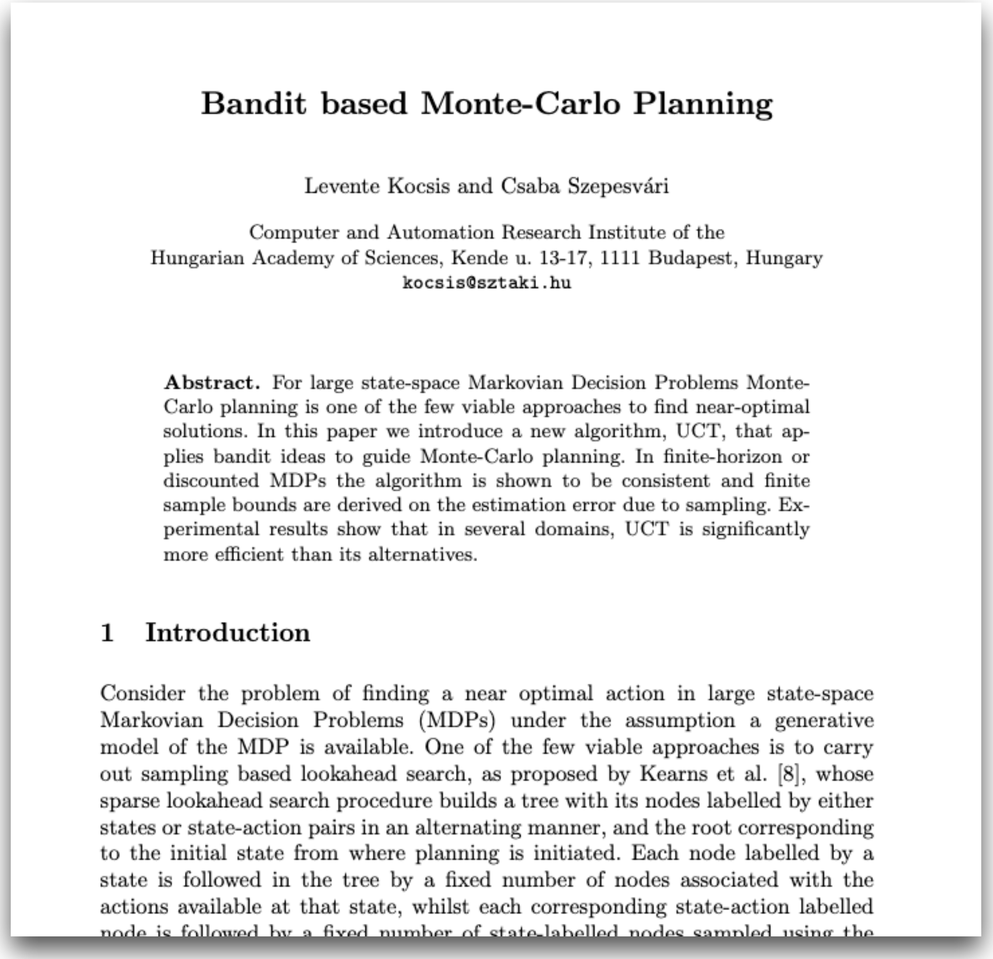


avrge constant contribution to regret per step.

found best arm and played it consistently after exploration.

linear because even though we found best arm, we still take random actions and accrue linear regret w.p. ε.

Applying Bandit Theory: Upper Confidence Trees (UCT)



Bandit based Monte-Carlo Planning

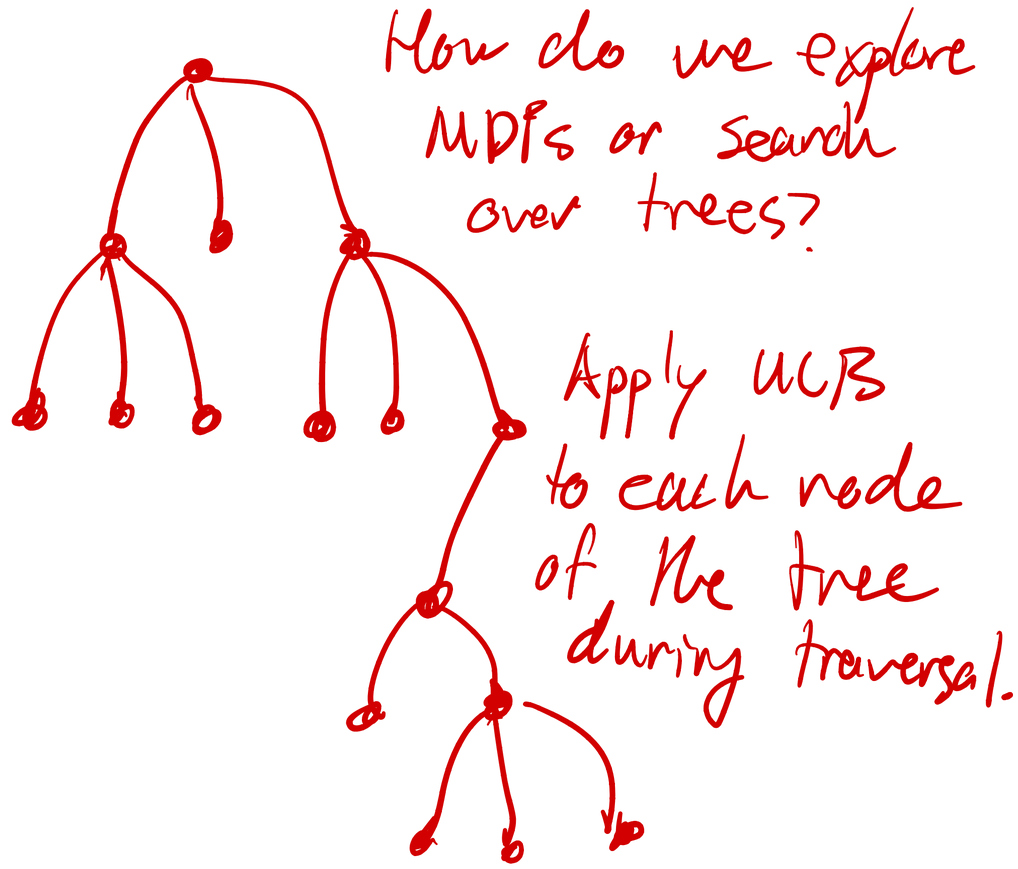
Levente Kocsis and Csaba Szepesvári

Computer and Automation Research Institute of the
Hungarian Academy of Sciences, Kende u. 13-17, 1111 Budapest, Hungary
kocsis@sztaki.hu

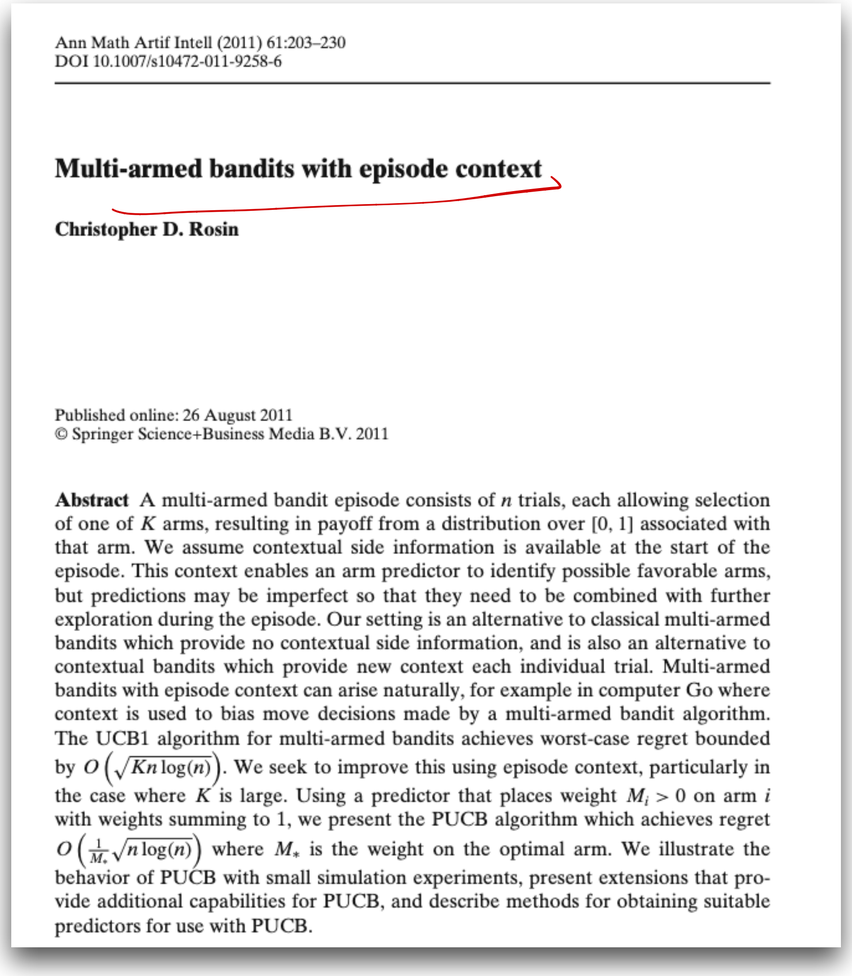
Abstract. For large state-space Markovian Decision Problems Monte-Carlo planning is one of the few viable approaches to find near-optimal solutions. In this paper we introduce a new algorithm, UCT, that applies bandit ideas to guide Monte-Carlo planning. In finite-horizon or discounted MDPs the algorithm is shown to be consistent and finite sample bounds are derived on the estimation error due to sampling. Experimental results show that in several domains, UCT is significantly more efficient than its alternatives.

1 Introduction

Consider the problem of finding a near optimal action in large state-space Markovian Decision Problems (MDPs) under the assumption a generative model of the MDP is available. One of the few viable approaches is to carry out sampling based lookahead search, as proposed by Kearns et al. [8], whose sparse lookahead search procedure builds a tree with its nodes labelled by either states or state-action pairs in an alternating manner, and the root corresponding to the initial state from where planning is initiated. Each node labelled by a state is followed in the tree by a fixed number of nodes associated with the actions available at that state, whilst each corresponding state-action labelled node is followed by a fixed number of state-labelled nodes sampled using the



PUCB: Predictor + UCB



Ann Math Artif Intell (2011) 61:203–230
DOI 10.1007/s10472-011-9258-6

Multi-armed bandits with episode context

Christopher D. Rosin

Published online: 26 August 2011
© Springer Science+Business Media B.V. 2011

Abstract A multi-armed bandit episode consists of n trials, each allowing selection of one of K arms, resulting in payoff from a distribution over [0, 1] associated with that arm. We assume contextual side information is available at the start of the episode. This context enables an arm predictor to identify possible favorable arms, but predictions may be imperfect so that they need to be combined with further exploration during the episode. Our setting is an alternative to classical multi-armed bandits which provide no contextual side information, and is also an alternative to contextual bandits which provide new context each individual trial. Multi-armed bandits with episode context can arise naturally, for example in computer Go where context is used to bias move decisions made by a multi-armed bandit algorithm. The UCB1 algorithm for multi-armed bandits achieves worst-case regret bounded by $O\left(\sqrt{Kn\log(n)}\right)$. We seek to improve this using episode context, particularly in the case where K is large. Using a predictor that places weight $M_i > 0$ on arm i with weights summing to 1, we present the PUCB algorithm which achieves regret $O\left(\frac{1}{M_*}\sqrt{n\log(n)}\right)$ where M_* is the weight on the optimal arm. We illustrate the behavior of PUCB with small simulation experiments, present extensions that provide additional capabilities for PUCB, and describe methods for obtaining suitable predictors for use with PUCB.

What if we also have some prior information to bias our exploration?

e.g. Neural-network policy that places some weights on each arm.

use the prior info to re-weight a UCB1-style exploration rule.

Two arms have means 0.9 and 0.1 (a large gap). Which algorithm takes advantage of this by exploring less?

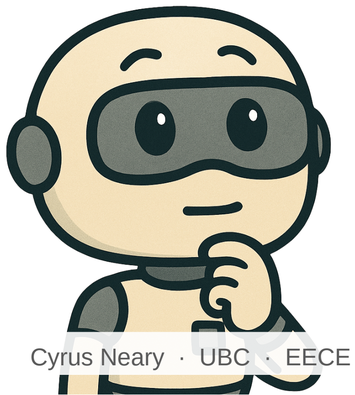
A: Uniform exploration with fixed N.

B: The ε-greedy algorithm.

C: Successive elimination.

D: All of them benefit equally.

E: I don’t understand.



Assume the *clean event* holds for a particular run. Is it possible for the successive elimination algorithm to remove the optimal arm a^* from consideration?

A: Yes, if its confidence interval is wider than the other arms'.
B: Yes, whenever its estimated mean falls below another arm's estimate.
C: Yes, because $UCB_t(a^*) \geq \mu^* \geq \mu(a') \geq LCB_t(a')$ for every a' at every t.
D: No, because $UCB_t(a^*) \geq \mu^* \geq \mu(a') \geq LCB_t(a')$ for every a' at every t.
E: I don't understand.